GUIA TÉCNICO

Ambientes Python e JupyterLab

Interpretadores, dependências, kernels, execução reproduzível e diagnóstico de problemas.

Propósito — servir como material de estudo aprofundado e referência rápida, com exemplos originais, atividades e integração com os notebooks.

Como utilizar — leia os conceitos, reproduza os exemplos no ambiente indicado e compare as atividades com o notebook correspondente. A consulta rápida aparece nas seções finais.

ROTEIRO DO DOCUMENTO

1. Mapa mental: as cinco camadas do ambiente
2. Ambientes virtuais: isolamento e limites
3. pip: instalação com rastreabilidade
4. Kernel e ambiente: relações que causam erros
5. JupyterLab: navegação e execução
6. Estado, execução determinística e reprodutibilidade
7. Arquivos, caminhos e diretórios
8. Problemas frequentes: árvore de decisão
9. Consulta rápida: procedimentos essenciais
10. Oficina guiada: construir um ambiente verificável
11. Exercícios de aprofundamento
12. Aprofundamento: resolução de dependências e compatibilidade
13. Aprofundamento: segurança, caminhos e publicação
14. Integração, leitura adicional e referências

1. Mapa mental: as cinco camadas do ambiente

Um projeto de Ciência de Dados depende de componentes distintos. **Sistema operacional** fornece processos e arquivos; **interpretador Python** executa instruções; **ambiente virtual** define um conjunto de pacotes; **kernel Jupyter** mantém o estado das células; e **JupyterLab** é a interface que se comunica com o kernel. Instalar o JupyterLab em um ambiente não obriga todos os notebooks a utilizarem aquele Python.

Camada	Responsabilidade	Como verificar
Python	Executa código e importa bibliotecas	`python --version`
Ambiente virtual	Isola bibliotecas e executáveis	`sys.prefix` e `sys.base_prefix`
pip	Instala distribuições para um Python	`python -m pip --version`
Kernel	Executa células, preserva variáveis	`sys.executable` no notebook
JupyterLab	Editor, explorador, consoles, terminal	Interface e seletor de kernel

Regra fundamental: antes de instalar ou atualizar uma biblioteca, identifique o interpretador que efetivamente executará o projeto. O terminal e o notebook podem estar conectados a ambientes diferentes.

Exemplo: investigar o Python ativo

```
import sys
from pathlib import Path
print('Executavel:', sys.executable)
print('Prefixo:', sys.prefix)
print('Base:', sys.base_prefix)
print('Pasta atual:', Path.cwd())
print('Em ambiente virtual?', sys.prefix != sys.base_prefix)
```

2. Ambientes virtuais: isolamento e limites

venv cria uma estrutura de diretórios que utiliza uma instalação de Python já existente. Os pacotes instalados nesse ambiente ficam separados dos pacotes de outros projetos, salvo configurações especiais. O ambiente não é uma máquina virtual, nem garante que código e bibliotecas produzam resultados idênticos em todos os sistemas operacionais.

Fluxo no Linux ou macOS

```
python -m venv .venv
source .venv/bin/activate
python --version
python -m pip --version
python -m pip check
deactivate
```

Fluxo no Windows com PowerShell

```
py -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip --version
deactivate
```

Se a política de execução impedir a ativação no PowerShell, consulte a configuração de segurança do computador. A ativação é um atalho: é possível executar diretamente `.venv/bin/python` ou `.venv\Scripts\python.exe` sem ativar o ambiente.

Não versionar ambientes virtuais: diretórios `.venv/`, `venv/` e cache de Python devem estar no `.gitignore`. Ambientes são recriados; o que deve ser documentado são os requisitos de instalação.

3. pip: instalação com rastreabilidade

`pip` instala distribuições Python e resolve dependências declaradas pelos pacotes. Usar `python -m pip` vincula o instalador ao interpretador escolhido, evitando confusões com o executável `pip` encontrado no PATH.

Comando	Finalidade
`python -m pip install polars`	Instalar uma distribuição
`python -m pip install -r requirements.txt`	Instalar dependências declaradas
`python -m pip show polars`	Inspecionar pacote, versão e caminho
`python -m pip list`	Listar distribuições instaladas
`python -m pip check`	Verificar incompatibilidades declaradas
`python -m pip freeze`	Gerar lista de pacotes e versões instaladas
`python -m pip uninstall polars`	Remover uma distribuição

Um exemplo de requisitos diretos

```
jupyterlab
ipykernel
ipywidgets
polars
pandas
numpy
matplotlib
```

`requirements.txt` com nomes sem versões é fácil de manter para exercícios educacionais, mas não fixa um ambiente. Para reprodutibilidade estrita, podem ser necessários versões fixadas, hashes, informações do sistema e controle de dependências transitivas. O resultado de `pip freeze` é uma fotografia do ambiente, não uma seleção curada dos pacotes necessários.

Evite atualizações indiscriminadas: modificar uma dependência pode alterar a API disponível no notebook. Teste primeiro em ambiente descartável e registre mudanças conscientes.

4. Kernel e ambiente: relações que causam erros

O kernel Python é o processo que recebe as células executadas no JupyterLab. Ele mantém objetos na memória até ser reiniciado. Um arquivo `.ipynb` contém células e metadados, mas não incorpora automaticamente todas as bibliotecas necessárias.

Registrar um kernel em um ambiente

```
python -m pip install ipykernel
python -m ipykernel install --user \
  --name ds-recursos --display-name 'Python (ds-recursos)'
jupyter kernelspec list
```

O nome `ds-recursos` identifica a especificação; o texto entre parênteses é o rótulo mostrado no seletor. Para usar um ambiente diferente, ative-o antes de instalar e registrar seu kernel.

Diagnóstico de divergência terminal × notebook

```
import sys
print(sys.executable)
```

No notebook, execute também `%pip show polars` em uma célula IPython para consultar os metadados do pacote associado ao kernel.

No IPython, `%pip` instala pacotes no ambiente do kernel em execução. Após instalar uma biblioteca que carrega módulos nativos ou altera dependências essenciais, pode ser necessário reiniciar o kernel.

Armadilha comum: executar uma célula com sucesso porque `x` já foi definido anteriormente. Reiniciar e executar tudo em ordem revela dependências ocultas entre células.

5. JupyterLab: navegação e execução

JupyterLab reúne gerenciador de arquivos, notebooks, editores de texto, consoles e terminais. A interface é separada do processo que executa o notebook, portanto um terminal aberto na interface pode não estar no mesmo ambiente do kernel.

Ação	Operação típica
Executar e avançar	`Shift+Enter`
Executar sem avançar	`Ctrl+Enter`
Inserir célula abaixo	`B` em modo comando
Inserir célula acima	`A` em modo comando
Alternar para Markdown	`M` em modo comando
Alternar para código	`Y` em modo comando
Salvar	`Ctrl+S`
Modo comando	`Esc`
Modo edição	`Enter`

Atalhos podem ser personalizados e variar entre sistemas ou configurações. Confira sempre o menu de comandos da instalação utilizada.

Células de código e Markdown

Células de código executam no kernel e podem produzir texto, tabelas, gráficos ou widgets. Células Markdown documentam decisões e apresentam resultados; Markdown não executa Python. O `execution_count` e a ordem física das células não garantem a mesma sequência de execução.

6. Estado, execução determinística e reprodutibilidade

Um notebook confiável deve poder ser executado de cima para baixo após reiniciar o kernel. Evite depender do estado que restou de execuções anteriores. Separe parâmetros, carga de dados, transformação, validação e geração de resultados.

Padrão simples de validação

```
from pathlib import Path
import numpy as np

rng = np.random.default_rng(123)
valores = rng.normal(loc=10, scale=2, size=100)
assert len(valores) == 100
print('Media:', round(float(valores.mean()), 2))
print('Saida em:', Path('outputs').resolve())
```

Uma semente controla o gerador usado neste exemplo, mas não garante determinismo de todas as bibliotecas, plataformas ou operações paralelas. Para resultados científicos, documente também entradas, dependências e parâmetros.

7. Arquivos, caminhos e diretórios

Caminhos relativos são interpretados a partir do diretório de trabalho do kernel, não necessariamente da localização física do arquivo `.ipynb`. Utilize `pathlib` e evite caminhos absolutos específicos de um computador.

```
from pathlib import Path
base = Path.cwd()
saidas = base / 'outputs'
saidas.mkdir(exist_ok=True)
arquivo = saidas / 'exemplo.txt'
arquivo.write_text('execucao concluida\n', encoding='utf-8')
print(arquivo.read_text(encoding='utf-8'))
```

O diretório `outputs/` pode conter arquivos reproduzíveis, mas resultados sensíveis ou volumosos precisam ser avaliados antes de um commit. No acervo, alguns CSVs/Parquet pequenos e didáticos foram deliberadamente preservados.

8. Problemas frequentes: árvore de decisão

Sintoma	Hipótese	Próximo passo
`ModuleNotFoundError`	Pacote não está no kernel ativo	Ver `sys.executable` e `%pip show nome`
Pacote instalado, import falha	Ambiente diferente ou sessão antiga	Selecionar kernel e reiniciar
Widget mostra texto	Integração frontend/backend falhou	Verificar `ipywidgets`, recarregar interface
Arquivo não encontrado	Diretório corrente diferente	Verificar `Path.cwd()`
Resultado muda sem alterar código	Estado anterior ou aleatoriedade	Reiniciar e executar em sequência
Instalação negada	Permissões ou sistema gerenciado	Criar ambiente isolado

Nunca instale pacotes como administrador como primeira tentativa para contornar erros. Investigue o ambiente, os caminhos e as permissões.

9. Consulta rápida: procedimentos essenciais

Objetivo	Comando ou ação
Criar ambiente	`python -m venv .venv`
Instalar requisitos	`python -m pip install -r requirements.txt`
Verificar dependências	`python -m pip check`
Iniciar JupyterLab	`jupyter lab`
Ver kernels	`jupyter kernelspec list`
Interpretador do notebook	`import sys; print(sys.executable)`
Localização atual	`from pathlib import Path; print(Path.cwd())`
Validar notebook	Reiniciar kernel e executar todas as células

10. Oficina guiada: construir um ambiente verificável

1. Crie um diretório de projeto e um ambiente `.venv`.
2. Instale `ipykernel`, `jupyterlab` e `numpy` com `python -m pip`.
3. Registre um kernel com nome identificável.
4. Crie um notebook com células de título, importações, parâmetro e análise.
5. Confira `sys.executable` e `Path.cwd()`; compare com o terminal.
6. Execute uma análise usando `numpy.random.default_rng` e uma semente explícita.
7. Reinicie o kernel e execute todas as células do início ao fim.
8. Registre os requisitos diretos e descreva o procedimento no README do projeto.

Critérios de aceite: as células executam em ordem; não há dependências de estado residual; os caminhos são portáveis; os requisitos correspondem às importações realmente utilizadas.

11. Exercícios de aprofundamento

1. Explique por que instalar uma biblioteca no terminal pode não resolver um import no JupyterLab.
2. Mostre dois métodos para descobrir o interpretador associado ao kernel.
3. Compare um ambiente virtual e um kernelspec: o que cada um controla?
4. Faça uma experiência executando células fora de ordem e depois corrigindo o notebook.
5. Produza uma checklist de reprodutibilidade para um projeto com dados locais.
6. Descreva o que `pip check` verifica e o que ele **não** valida.

12. Aprofundamento: resolução de dependências e compatibilidade

A instalação de um pacote envolve pelo menos três decisões: qual **distribuição** instalar, quais versões satisfazem as restrições e qual interpretador receberá os arquivos. Nome de distribuição e nome importado podem diferir: a instalação é feita com o identificador do pacote, mas a importação usa o nome do módulo exposto. Por isso, um erro de importação nem sempre significa que o comando `pip install` foi escrito corretamente.

Dependência direta é declarada pelo projeto; **dependência transitiva** é exigida por uma biblioteca que o projeto utiliza. Atualizar uma dependência direta pode alterar várias transitivas, mesmo sem modificar `requirements.txt`. A solução deve considerar sistema operacional, arquitetura, versão de Python e disponibilidade de pacotes binários compatíveis.

Verificação	Interpretação
`python -m pip list`	Mostra o estado atual, inclusive transitivas
`python -m pip check`	Busca incompatibilidades declaradas de pacotes instalados
`python -m pip show nome`	Localização e metadados de uma distribuição
`python -c "import modulo"`	Testa importação com um interpretador específico
Reiniciar kernel e executar tudo	Detecta dependência de estado residual

Diagnóstico guiado: se `import polars` falhar no notebook, comece imprimindo `sys.executable`. Execute o pip correspondente ou `%pip` no notebook, confirme a instalação e reinicie o kernel. Se persistir, investigue mensagens de versão e bibliotecas compartilhadas, em vez de instalar pacotes repetidamente em vários ambientes.

13. Aprofundamento: segurança, caminhos e publicação

O histórico de um notebook pode conter resultados antigos, credenciais copiadas para células e caminhos completos de usuário. O fato de um arquivo estar listado no `.gitignore` não remove conteúdo já rastreado pelo Git. Antes de publicar, analise o histórico de mudanças e as saídas salvas.

```
# Veja quais arquivos seriam incluídos em um commit
git status --short
# Inspecione somente as alterações preparadas
git diff --cached --stat
git diff --cached --check
```

Evite `git add .` em repositórios educacionais com datasets, outputs e PDFs heterogêneos. Prefira adicionar explicitamente os arquivos autorizados e revisar o índice antes do commit. Tokens devem ficar em gerenciadores de credenciais ou configurações apropriadas, nunca em um notebook publicado.

Estratégia de verificação para compartilhar uma análise

1. A entrada do experimento está identificada e é redistribuível?
2. Os comandos criam um ambiente limpo sem modificar arquivos originais?
3. O kernel corresponde ao interpretador com dependências declaradas?
4. Executar tudo após reiniciar produz saídas coerentes?
5. Tabelas e gráficos não revelam informações pessoais?
6. O README explica instalação, execução e limitações?

Caso de teste: compartilhe um notebook simples com uma segunda pessoa ou em ambiente virtual novo. Se a execução exigir passos que não estão documentados, o projeto ainda não está suficientemente reproduzível.

14. Integração, leitura adicional e referências

Notebook correspondente: `01_Ambiente_Python_e_JupyterLab.ipynb`. Use-o para aplicar os comandos e comparar comportamentos no seu ambiente, mantendo o notebook original preservado.

- Python venv: https://docs.python.org/3/library/venv.html
- Python packaging: https://packaging.python.org/
- pip: https://pip.pypa.io/en/stable/
- JupyterLab: https://jupyterlab.readthedocs.io/
- ipykernel: https://ipython.readthedocs.io/en/stable/install/kernel_install.html
- NumPy random: https://numpy.org/doc/stable/reference/random/index.html

Este documento é autoral; materiais rápidos de terceiros serviram apenas como referência temática. Verifique sempre a documentação oficial para APIs sujeitas a mudança.